Quantium Virtual Internship

Retail Strategy & Analytics — Task 2

Evaluating the impact of a store trial in stores 77, 86 and 88 using a control-store experimental design.

Prepared by **Fariba Kazi**

Prepared for the Chips Category Manager | Analysis in Python

1. Approach

The client ran a layout trial in three stores (77, 86, 88) from February to April 2019. To isolate the trial effect we pair each trial store with a **control store**: an established store (operating across the full 12-month observation period) whose pre-trial behaviour most closely resembles the trial store. Comparing the trial store against its scaled control during the trial period lets us attribute any uplift to the trial rather than to seasonality or market trends.

For every store and month we computed five measures: total sales, number of customers, transactions per customer, chips per transaction, and average price per unit. Control selection uses two of these drivers — **monthly total sales** and **monthly customer count**.

2. Control-store selection metric

Each candidate store is scored against the trial store on each driver using two complementary measures, computed over the pre-trial months only:

Pearson correlation — captures whether the two stores move *together* month to month (shape of the trend).
Magnitude distance — a 0–1 score from the absolute level difference, standardised as 1 − (distance − min)/(max − min), so a score near 1 means the two stores sit at a *similar level*.

For each driver we average the two into a driver score (equal 0.5 / 0.5 weight on correlation vs magnitude). The two driver scores are then averaged into a final score. The non-trial store with the highest final score becomes the control. Reusable functions `calculateCorrelation` and `calculateMagnitudeDistance` do this work so the routine is applied identically to all three trial stores.

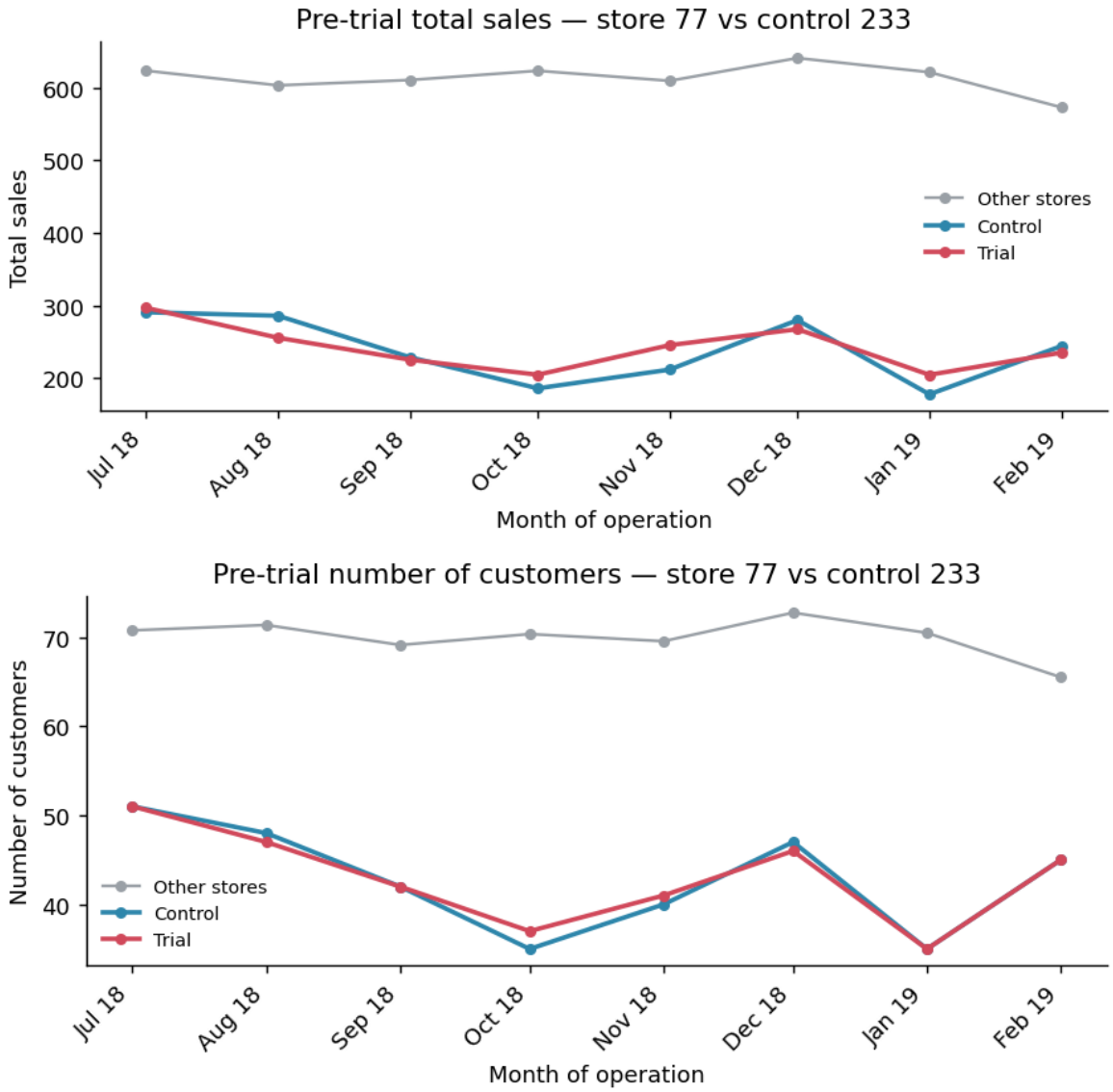
Trial store	Selected control store
77	233
86	155
88	237

3. Assessing the trial

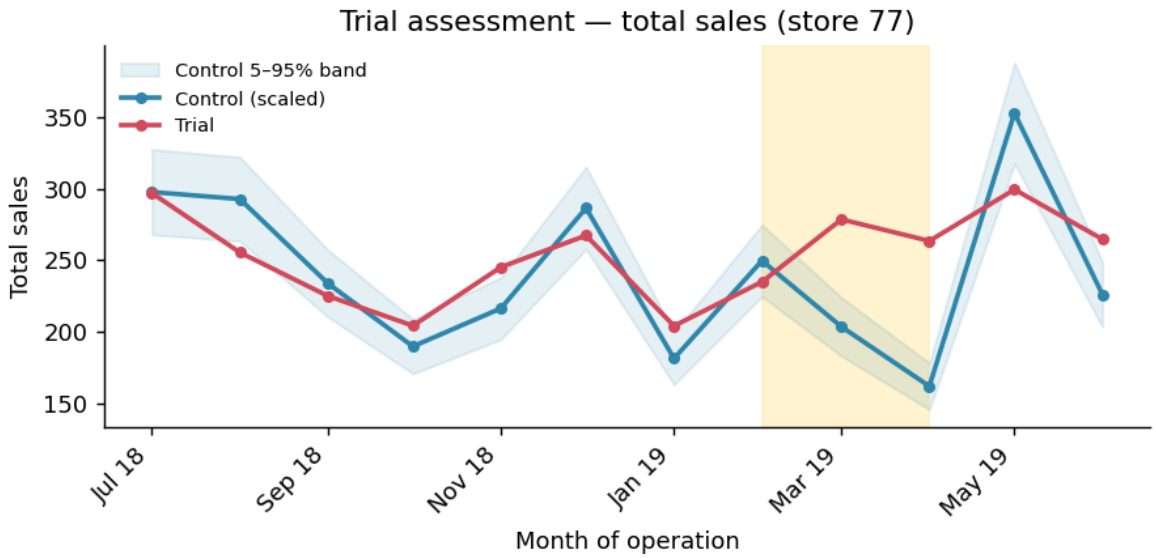
We scale the control store's pre-trial sales (and customers) to match the trial store's level, then measure the percentage difference each month. Under the null hypothesis the trial period behaves like the pre-trial period, so we take the standard deviation of the pre-trial percentage differences and compute a t-statistic for each trial month with 7 degrees of freedom. The 95th-percentile critical value is **t = 1.895**. A trial month whose t-statistic exceeds this — equivalently, a trial point falling outside the control's 5–95% confidence band — is a statistically significant uplift.

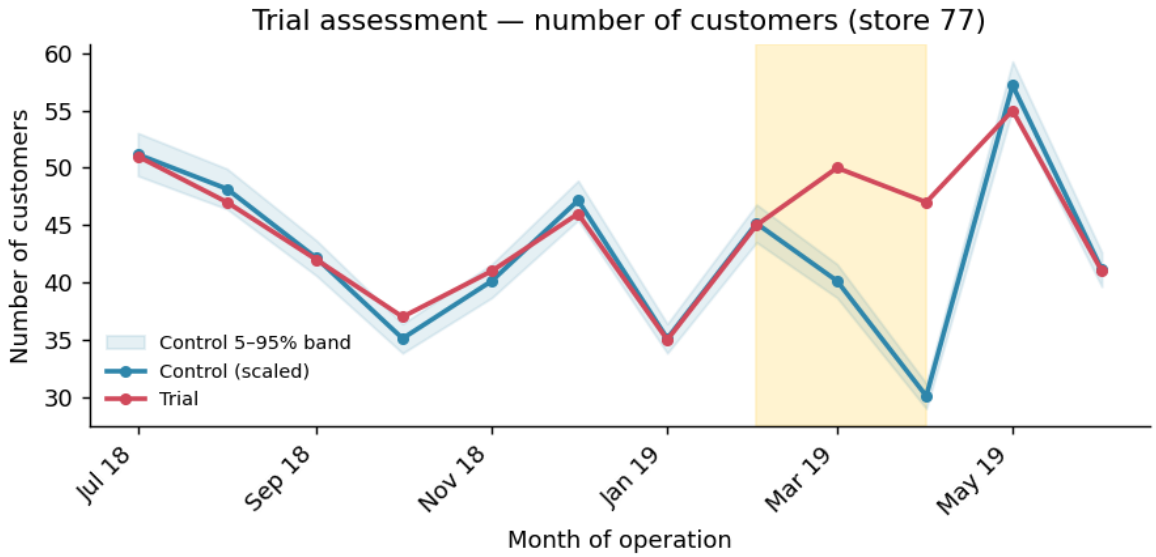
Trial store 77 (control store 233)

Pre-trial check. Before trusting the comparison we confirm the trial and control stores tracked each other closely *before* the trial on both drivers:



Trial assessment. The shaded band marks the Feb–Apr 2019 trial window; the blue ribbon is the control's 5–95% confidence interval.





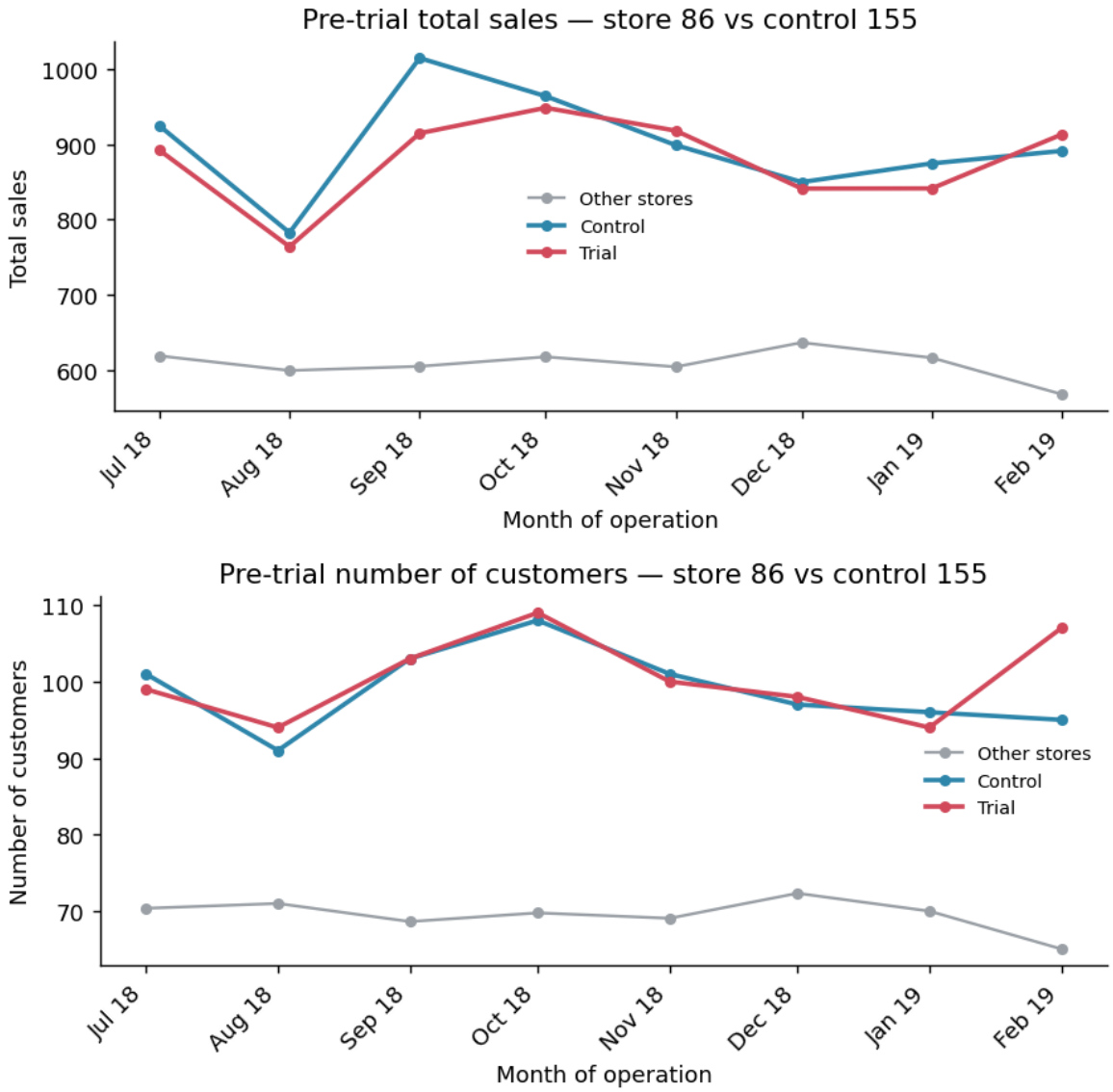
Sales t-statistics: Feb t=-1.18 (not significant); Mar t=7.34 (significant); Apr t=12.48 (significant).

Customer t-statistics: Feb t=-0.18 (not significant); Mar t=13.48 (significant); Apr t=30.78 (significant).

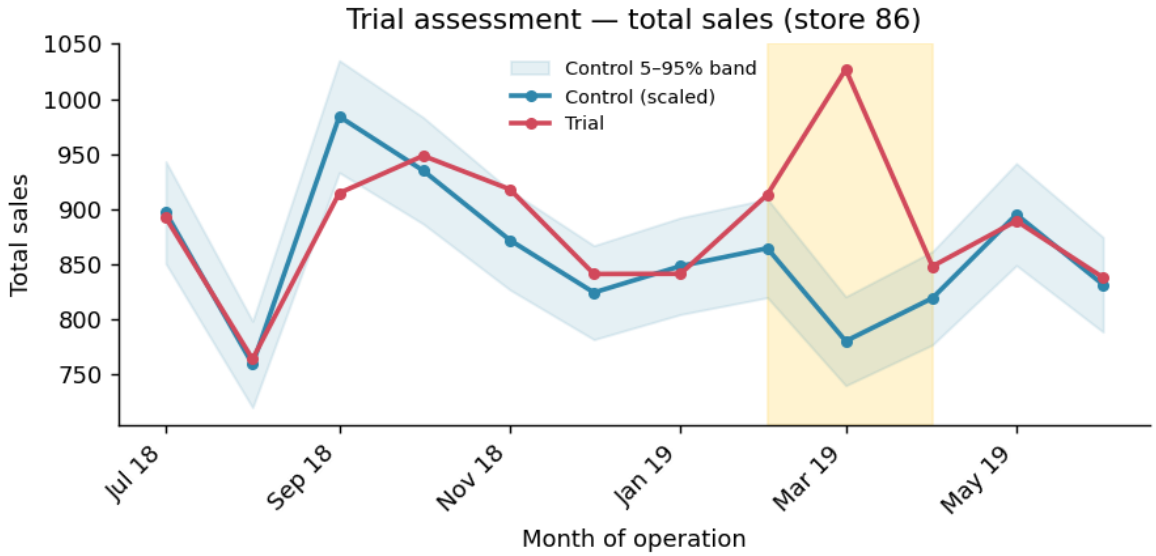
Store 77 — significant sales uplift. Sales in the trial store sit clearly above the control's 95% band in March and April, and customer counts are significantly higher across the same months. The trial drove both more customers and more sales. **Conclusion: the trial was successful in store 77.**

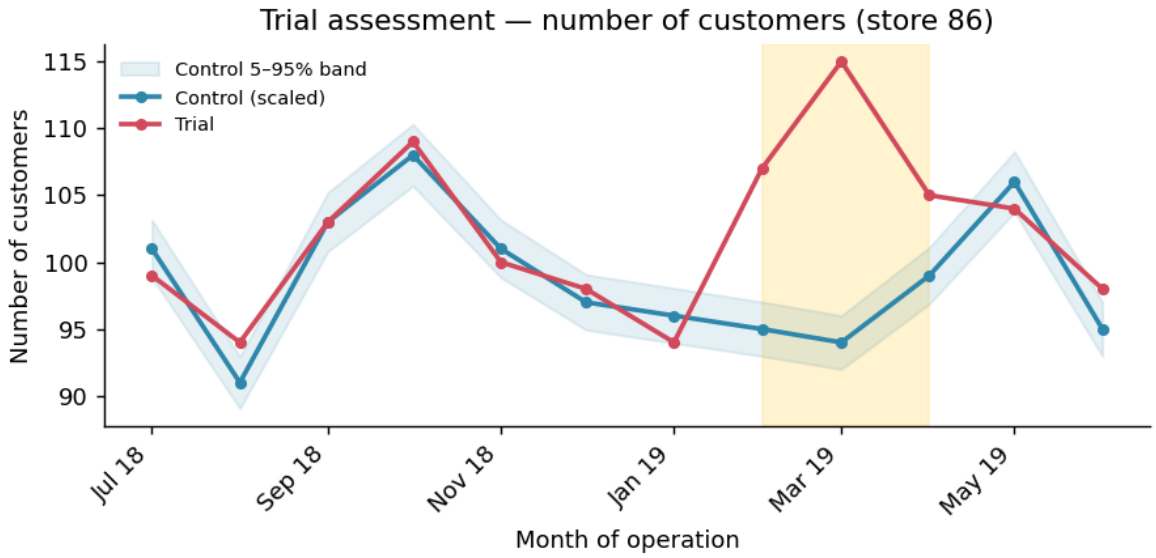
Trial store 86 (control store 155)

Pre-trial check. Before trusting the comparison we confirm the trial and control stores tracked each other closely *before* the trial on both drivers:



Trial assessment. The shaded band marks the Feb–Apr 2019 trial window; the blue ribbon is the control's 5–95% confidence interval.





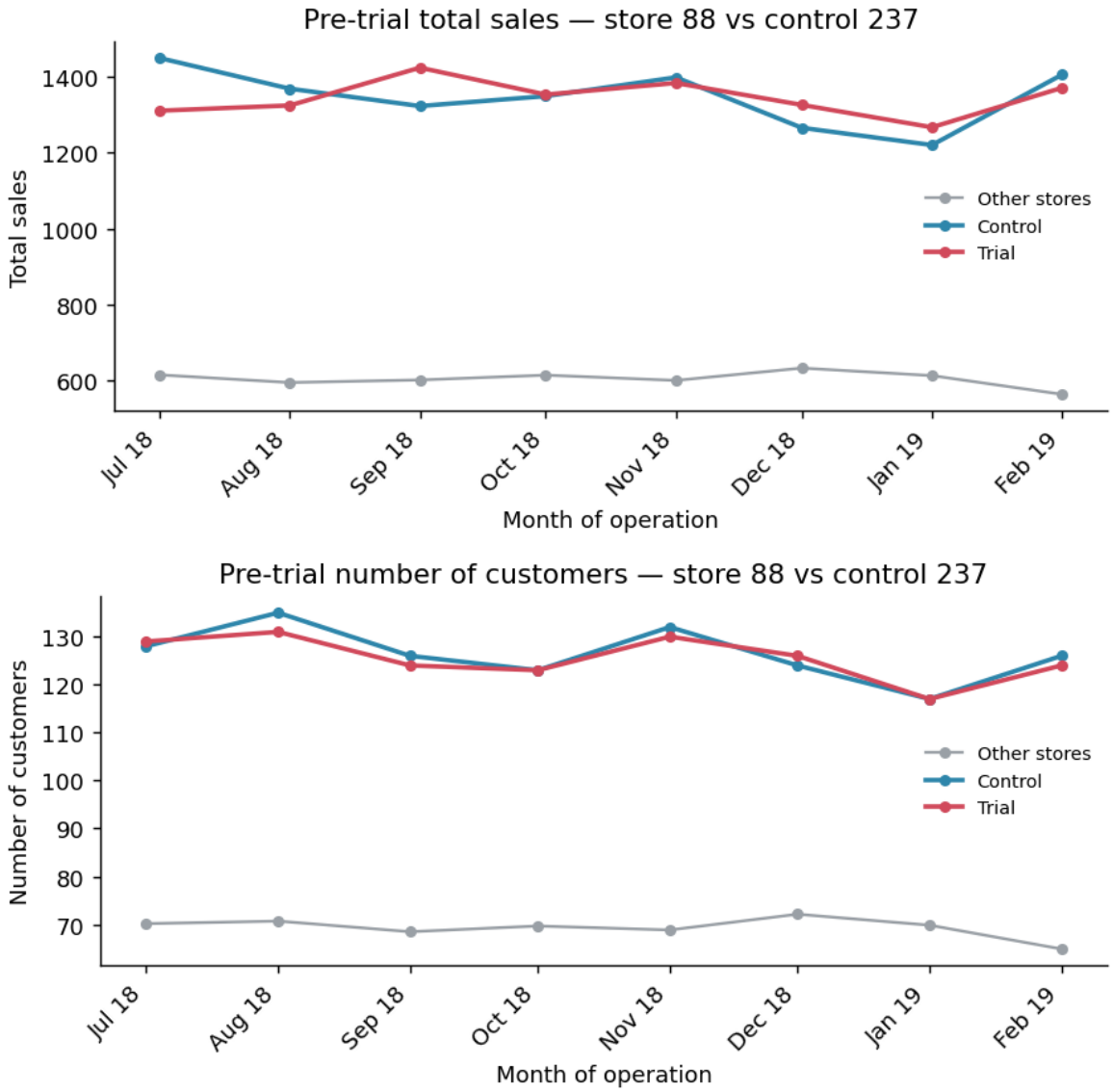
Sales t-statistics: Feb t=2.18 (significant); Mar t=12.23 (significant); Apr t=1.36 (not significant).

Customer t-statistics: Feb t=11.82 (significant); Mar t=20.90 (significant); Apr t=5.67 (significant).

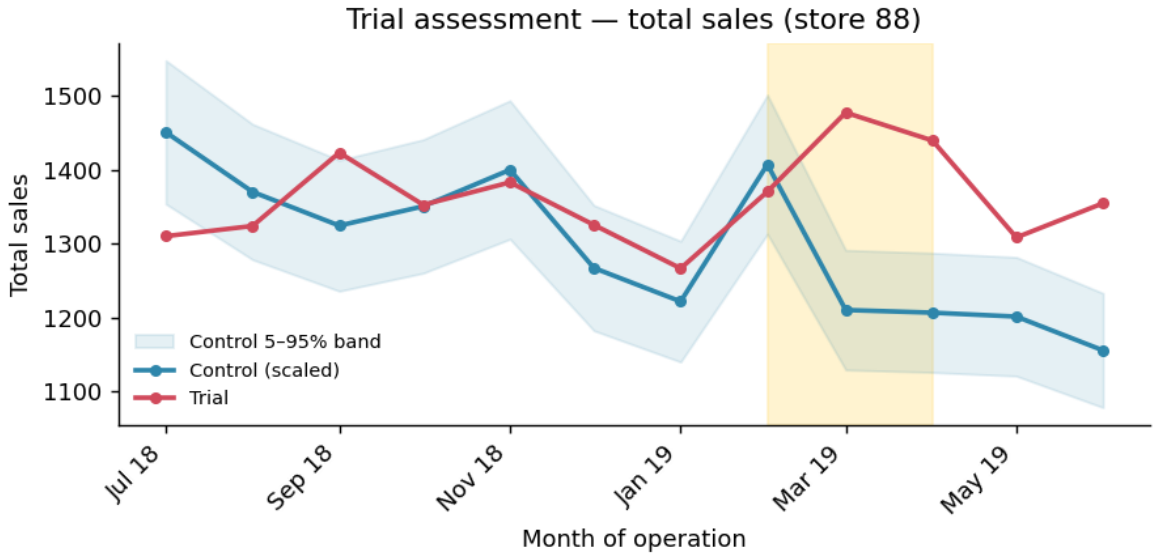
Store 86 — customers up, sales mixed. Customer counts are significantly higher in all three trial months, but sales only break the 95% band in one month — sitting inside the confidence band otherwise. This suggests the layout attracted more shoppers without a matching rise in spend, possibly due to lower prices or promotions. **Conclusion: a positive customer effect, but the sales impact is not conclusive — worth checking with the Category Manager whether the trial was implemented differently here.**

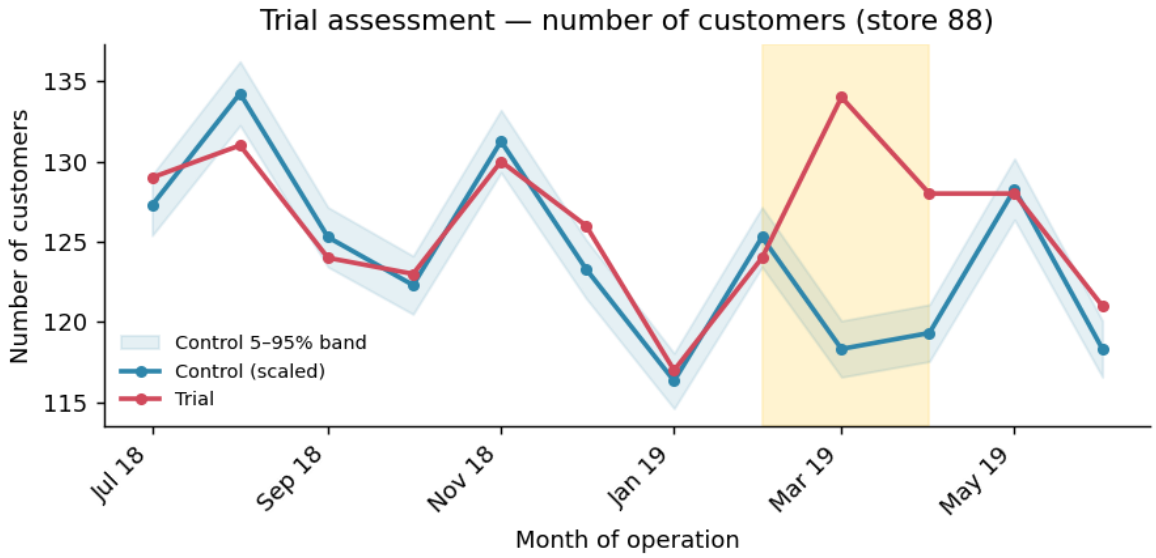
Trial store 88 (control store 237)

Pre-trial check. Before trusting the comparison we confirm the trial and control stores tracked each other closely *before* the trial on both drivers:



Trial assessment. The shaded band marks the Feb–Apr 2019 trial window; the blue ribbon is the control's 5–95% confidence interval.





Sales t-statistics: Feb t=-0.78 (not significant); Mar t=6.60 (significant); Apr t=5.77 (significant).

Customer t-statistics: Feb t=-1.39 (not significant); Mar t=17.87 (significant); Apr t=9.81 (significant).

Store 88 — significant sales uplift. Both sales and customer counts exceed the control's 95% band in two of the three trial months, indicating a clear positive trial effect. **Conclusion: the trial was successful in store 88.**

4. Conclusion & recommendation

Control stores 233, 155 and 237 were matched to trial stores 77, 86 and 88 respectively. Trial stores 77 and 88 show a statistically significant increase in sales in at least two of the three trial months, driven by a clear rise in customer numbers. Store 86 shows a strong, significant increase in customers but a sales response that is not conclusive, which may reflect a different implementation or promotional pricing in that location.

Recommendation: the trial layout has a positive and significant impact overall and is a good candidate for a wider roll-out. We recommend confirming with the client why store 86 behaved differently before finalising the rollout plan, then preparing the findings for presentation to the Category Manager.

Appendix — Python code

Full analysis script (pandas / scipy / matplotlib).

```
"""
Quantium Virtual Internship - Retail Strategy and Analytics - Task 2
Trial vs Control store analysis (Python implementation)
"""
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import os

plt.rcParams.update({
    "figure.dpi": 130,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 10,
})

OUT = "/home/claude/figs"
os.makedirs(OUT, exist_ok=True)

# ---------------------------------------------------------------------
# 1. Load data
# ---------------------------------------------------------------------
data = pd.read_csv("/mnt/user-data/uploads/QVI_data__1_.csv")
data["DATE"] = pd.to_datetime(data["DATE"])

# YEARMONTH as yyyymm integer
data["YEARMONTH"] = data["DATE"].dt.year * 100 + data["DATE"].dt.month

# ---------------------------------------------------------------------
# 2. Build monthly measures per store
#    totSales, nCustomers, nTxnPerCust, nChipsPerTxn, avgPricePerUnit
# ---------------------------------------------------------------------
def make_measures(df):
    g = df.groupby(["STORE_NBR", "YEARMONTH"])
    m = g.agg(
        totSales=("TOT_SALES", "sum"),
        nCustomers=("LYLTY_CARD_NBR", "nunique"),
        nTxn=("TXN_ID", "nunique"),
        totQty=("PROD_QTY", "sum"),
    ).reset_index()
    m["nTxnPerCust"] = m["nTxn"] / m["nCustomers"]
    m["nChipsPerTxn"] = m["totQty"] / m["nTxn"]
    m["avgPricePerUnit"] = m["totSales"] / m["totQty"]
    return m.sort_values(["STORE_NBR", "YEARMONTH"]).reset_index(drop=True)

measureOverTime = make_measures(data)

# Stores observed for full 12 months
counts = measureOverTime.groupby("STORE_NBR").size()
storesWithFullObs = counts[counts == 12].index
preTrial = measureOverTime[
    (measureOverTime["YEARMONTH"] < 201902)
    & (measureOverTime["STORE_NBR"].isin(storesWithFullObs))
].copy()

# ---------------------------------------------------------------------
# 3. Functions: correlation & magnitude distance
# ---------------------------------------------------------------------
def calculateCorrelation(inputTable, metricCol, trial_store):
    rows = []
    trial_vec = inputTable[inputTable["STORE_NBR"] == trial_store].set_index("YEARMONTH")[metricCol]
    for s in inputTable["STORE_NBR"].unique():
        cand = inputTable[inputTable["STORE_NBR"] == s].set_index("YEARMONTH")[metricCol]
        joined = pd.concat([trial_vec, cand], axis=1, join="inner")
        joined.columns = ["a", "b"]
        if len(joined) > 1 and joined["a"].std() > 0 and joined["b"].std() > 0:
            corr = joined["a"].corr(joined["b"])
        else:
            corr = 0.0
        rows.append({"Store1": trial_store, "Store2": s, "corr_measure": corr})
    return pd.DataFrame(rows)

def calculateMagnitudeDistance(inputTable, metricCol, trial_store):
    trial_vec = inputTable[inputTable["STORE_NBR"] == trial_store].set_index("YEARMONTH")[metricCol]
    recs = []
    for s in inputTable["STORE_NBR"].unique():
        cand = inputTable[inputTable["STORE_NBR"] == s].set_index("YEARMONTH")[metricCol]
        joined = pd.concat([trial_vec, cand], axis=1, join="inner")
        joined.columns = ["a", "b"]
        for ym, r in joined.iterrows():
            recs.append({"Store1": trial_store, "Store2": s,
                         "YEARMONTH": ym, "measure": abs(r["a"] - r["b"])})
    dist = pd.DataFrame(recs)
    # standardise per Store1, YEARMONTH so it ranges 0..1
    mm = dist.groupby(["Store1", "YEARMONTH"])["measure"].agg(minDist="min", maxDist="max").reset_index()
    dist = dist.merge(mm, on=["Store1", "YEARMONTH"])
```

```
    rng = (dist["maxDist"] - dist["minDist"]).replace(0, np.nan)
    dist["magnitudeMeasure"] = 1 - (dist["measure"] - dist["minDist"]) / rng
    dist["magnitudeMeasure"] = dist["magnitudeMeasure"].fillna(1.0)
    out = dist.groupby(["Store1", "Store2"])["magnitudeMeasure"].mean().reset_index()
    out.columns = ["Store1", "Store2", "mag_measure"]
    return out

# ------------------------------------------------------------------------
# 4. Control-store selection routine
# ------------------------------------------------------------------------
def select_control(trial_store, corr_weight=0.5):
    cs = calculateCorrelation(preTrial, "totSales", trial_store)
    cc = calculateCorrelation(preTrial, "nCustomers", trial_store)
    ms = calculateMagnitudeDistance(preTrial, "totSales", trial_store)
    mc = calculateMagnitudeDistance(preTrial, "nCustomers", trial_store)

    sN = cs.merge(ms, on=["Store1", "Store2"])
    sN["scoreNSales"] = corr_weight * sN["corr_measure"] + (1 - corr_weight) * sN["mag_measure"]
    sC = cc.merge(mc, on=["Store1", "Store2"])
    sC["scoreNCust"] = corr_weight * sC["corr_measure"] + (1 - corr_weight) * sC["mag_measure"]

    score = sN[["Store1", "Store2", "scoreNSales"]].merge(
        sC[["Store1", "Store2", "scoreNCust"]], on=["Store1", "Store2"])
    score["finalControlScore"] = 0.5 * score["scoreNSales"] + 0.5 * score["scoreNCust"]
    ranked = score[score["Store2"] != trial_store].sort_values("finalControlScore", ascending=False)
    return int(ranked.iloc[0]["Store2"]), score

# ------------------------------------------------------------------------
# Helper: transaction-month axis date
# ------------------------------------------------------------------------
def ym_to_date(ym):
    return pd.to_datetime(f"{ym // 100}-{ym % 100:02d}-01")

# ------------------------------------------------------------------------
# 5. Pre-trial driver visual check
# ------------------------------------------------------------------------
def pretrial_plot(trial_store, control_store, metric, ylabel, fname):
    df = measureOverTime.copy()
    df["Store_type"] = np.where(df["STORE_NBR"] == trial_store, "Trial",
                        np.where(df["STORE_NBR"] == control_store, "Control", "Other stores"))
    grp = df.groupby(["YEARMONTH", "Store_type"])[metric].mean().reset_index()
    grp = grp[grp["YEARMONTH"] < 201903]
    grp["TransactionMonth"] = grp["YEARMONTH"].apply(ym_to_date)

    fig, ax = plt.subplots(figsize=(7, 3.5))
    colors = {"Trial": "#d1495b", "Control": "#2e86ab", "Other stores": "#9aa0a6"}
    for st in ["Other stores", "Control", "Trial"]:
        sub = grp[grp["Store_type"] == st].sort_values("TransactionMonth")
        ax.plot(sub["TransactionMonth"], sub[metric], marker="o", ms=4,
                label=st, color=colors[st], lw=2 if st != "Other stores" else 1.3)
    ax.set_xlabel("Month of operation"); ax.set_ylabel(ylabel)
    ax.set_title(f"Pre-trial {ylabel.lower()} — store {trial_store} vs control {control_store}")
    ax.legend(frameon=False, fontsize=8)
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %y"))
    fig.autofmt_xdate(rotation=45); fig.tight_layout()
    fig.savefig(f"{OUT}/{fname}", bbox_inches="tight"); plt.close(fig)

# ------------------------------------------------------------------------
# 6. Trial assessment (sales or customers) with 5/95 CI band
# ------------------------------------------------------------------------
def assess_trial(trial_store, control_store, metric, ylabel, fname):
    pre = preTrial
    scaling = (pre[(pre.STORE_NBR == trial_store) & (pre.YEARMONTH < 201902)][metric].sum()
               / pre[(pre.STORE_NBR == control_store) & (pre.YEARMONTH < 201902)][metric].sum())

    mot = measureOverTime.copy()
    scaledControl = mot[mot.STORE_NBR == control_store][["YEARMONTH", metric]].copy()
    scaledControl["controlScaled"] = scaledControl[metric] * scaling

    trial = mot[mot.STORE_NBR == trial_store][["YEARMONTH", metric]].copy()
    pdiff = scaledControl.merge(trial, on="YEARMONTH", suffixes=("_ctrl", "_trial"))
    pdiff["percentageDiff"] = (np.abs(pdiff["controlScaled"] - pdiff[f"{metric}_trial"])
                               / pdiff["controlScaled"])

    stdDev = pdiff[pdiff.YEARMONTH < 201902]["percentageDiff"].std()
    dof = 7
    # t-values for trial months
    pdiff["tValue"] = (pdiff[f"{metric}_trial"] - pdiff["controlScaled"]) / pdiff["controlScaled"] / stdDev
    tcrit = stats.t.ppf(0.95, dof)
    trial_months = pdiff[(pdiff.YEARMONTH > 201901) & (pdiff.YEARMONTH < 201905)]

    # Build plot table: trial, control (scaled), control +/- 2 stdDev band
    plot = pdiff[["YEARMONTH"]].copy()
    plot["Trial"] = pdiff[f"{metric}_trial"]
    plot["Control"] = pdiff["controlScaled"]
    plot["Control 95th %"] = pdiff["controlScaled"] * (1 + stdDev * 2)
    plot["Control 5th %"] = pdiff["controlScaled"] * (1 - stdDev * 2)
    plot["TransactionMonth"] = plot["YEARMONTH"].apply(ym_to_date)
    plot = plot.sort_values("TransactionMonth")

    fig, ax = plt.subplots(figsize=(7, 3.5))
    # trial period shading Feb-Apr 2019
    ax.axvspan(ym_to_date(201902), ym_to_date(201904), color="#ffe8a3", alpha=0.5)
```

```
    ax.fill_between(plot["TransactionMonth"], plot["Control 5th %"], plot["Control 95th %"],
                    color="#2e86ab", alpha=0.12, label="Control 5–95% band")
    ax.plot(plot["TransactionMonth"], plot["Control"], color="#2e86ab", lw=2, marker="o", ms=4, label="Control (scaled)")
    ax.plot(plot["TransactionMonth"], plot["Trial"], color="#d1495b", lw=2, marker="o", ms=4, label="Trial")
    ax.set_xlabel("Month of operation"); ax.set_ylabel(ylabel)
    ax.set_title(f"Trial assessment — {ylabel.lower()} (store {trial_store})")
    ax.legend(frameon=False, fontsize=8)
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %y"))
    fig.autofmt_xdate(rotation=45); fig.tight_layout()
    fig.savefig(f"{OUT}/{fname}", bbox_inches="tight"); plt.close(fig)

    return {
        "scaling": scaling, "stdDev": stdDev, "tcrit": tcrit,
        "trial_months": trial_months[["YEARMONTH", "tValue"]].to_dict("records"),
    }

# ---------------------------------------------------------------------------
# Run for all three trial stores
# ---------------------------------------------------------------------------
results = {}
for trial_store in [77, 86, 88]:
    control_store, _ = select_control(trial_store)
    pretrial_plot(trial_store, control_store, "totSales", "Total sales",
                  f"pre_sales_{trial_store}.png")
    pretrial_plot(trial_store, control_store, "nCustomers", "Number of customers",
                  f"pre_cust_{trial_store}.png")
    sales_res = assess_trial(trial_store, control_store, "totSales", "Total sales",
                             f"trial_sales_{trial_store}.png")
    cust_res = assess_trial(trial_store, control_store, "nCustomers", "Number of customers",
                            f"trial_cust_{trial_store}.png")
    results[trial_store] = {"control": control_store, "sales": sales_res, "cust": cust_res}
    print(f"Trial {trial_store} -> control {control_store}")
    print("  sales tcrit=%.3f" % sales_res["tcrit"])
    for r in sales_res["trial_months"]:
        print(f"    sales {r['YEARMONTH']}: t={r['tValue']:.3f}")
    for r in cust_res["trial_months"]:
        print(f"    cust  {r['YEARMONTH']}: t={r['tValue']:.3f}")

import json
with open("/home/claude/results.json", "w") as f:
    json.dump(results, f, indent=2, default=str)
print("DONE")
```